Customer Shopping Behaviour Analysis

1. Project Overview

This project analyses customer shopping behaviour using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behaviour to guide strategic business decisions.

2. Dataset Summary

- Rows: 3,900
- Columns: 18
- Key Features:
  Customer demographics (Age, Gender, Location, Subscription Status)
  Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Colour)
  Shopping behaviour (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)
- Missing Data: 37 values in Review Rating column.

3. Exploratory Data Analysis using Python

First of all, I did data preparation and cleaning in Python:

- Data Loading: Imported the dataset using pandas.
- Used df.info() to check structure and .describe() for summary statistics.

Load Dataset ¶

```
[2]: df = pd.read_csv(r"C:\Users\bittu kumar\OneDrive\Documents\1_project\customer_shopping_behavior.csv")
```

```
[3]: df.head()
```

[3]:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Payment Method	Frequency of Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	Venmo	Fortnightly
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	Cash	Fortnightly
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	PayPal	Annually

[4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   str
 3   Item Purchased          3900 non-null   str
 4   Category                3900 non-null   str
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   str
 7   Size                    3900 non-null   str
 8   Color                   3900 non-null   str
 9   Season                  3900 non-null   str
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   str
 12  Shipping Type           3900 non-null   str
 13  Discount Applied        3900 non-null   str
 14  Promo Code Used         3900 non-null   str
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   str
 17  Frequency of Purchases  3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB
```

[5]:
```
df.describe()
```

[5]:

	Customer ID	Age	Purchase Amount (USD)	Review Rating	Previous Purchases
count	3900.000000	3900.000000	3900.000000	3863.000000	3900.000000
mean	1950.500000	44.068462	59.764359	3.750065	25.351538
std	1125.977353	15.207589	23.685392	0.716983	14.447125
min	1.000000	18.000000	20.000000	2.500000	1.000000
25%	975.750000	31.000000	39.000000	3.100000	13.000000
50%	1950.500000	44.000000	60.000000	3.800000	25.000000
75%	2925.250000	57.000000	81.000000	4.400000	38.000000
max	3900.000000	70.000000	100.000000	5.000000	50.000000

- Handled missing values: Checked for null values and filled missing values in the Review Rating column using the median rating of each product category.

```
[6]: df.isnull().sum()
```

```
[6]: Customer ID                0
     Age                        0
     Gender                     0
     Item Purchased             0
     Category                   0
     Purchase Amount (USD)      0
     Location                   0
     Size                       0
     Color                      0
     Season                     0
     Review Rating             37
     Subscription Status        0
     Shipping Type              0
     Discount Applied           0
     Promo Code Used            0
     Previous Purchases         0
     Payment Method             0
     Frequency of Purchases     0
     dtype: int64
```

```
[7]: df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))
```

```
[8]: df.isnull().sum()
```

```
[8]: Customer ID                0
     Age                        0
     Gender                     0
     Item Purchased             0
     Category                   0
     Purchase Amount (USD)      0
     Location                   0
     Size                       0
     Color                      0
     Season                     0
     Review Rating              0
     Subscription Status        0
     Shipping Type              0
     Discount Applied           0
     Promo Code Used            0
     Previous Purchases         0
     Payment Method             0
     Frequency of Purchases     0
     dtype: int64
```

- Column Standardization: Renamed columns to snake case for better readability and documentation.

```
[9]: df.columns = df.columns.str.lower()
     df.columns = df.columns.str.replace(" ","_")
```

```
[10]: df.columns
```

```
[10]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount_(usd)', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='str')
```

```
[11]: df = df.rename(columns = {'purchase_amount_(usd)': 'purchase_amount'})
```

```
[12]: df.columns
```

```
[12]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='str')
```

- Created age group column by binning customer ages.

```
[13]: # create a new column age_group

      labels = ['Young Audlt', 'Adult', 'Middle-aged', 'Senior']

      df['age_group'] = pd.qcut(df['age'], q = 4, labels = labels )
```

```
[14]: df[['age', 'age_group']].head()
```

[14]:

	age	age_group
0	55	Middle-aged
1	19	Young Audlt
2	50	Middle-aged
3	21	Young Audlt
4	45	Middle-aged

- Created purchase frequency days column from purchase data.

```
[15]: # create a column purchase_frequency_days

frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}

df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)
```

```
[16]: df[['frequency_of_purchases', 'purchase_frequency_days']].head()
```

[16]:

	frequency_of_purchases	purchase_frequency_days
0	Fortnightly	14
1	Fortnightly	14
2	Weekly	7
3	Weekly	7
4	Annually	365

- Data Consistency Check: Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

```
[18]: (df['discount_applied'] ==  df['promo_code_used']).all()
```

```
[18]: np.True_
```

```
[19]: # since both the column are same, we can drop promo_code_used column

df = df.drop('promo_code_used', axis = 1)
```

```
[20]: df.columns
```

```
[20]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='str')
```

- Database Integration: Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis.

```
[21]: from sqlalchemy import create_engine
```

```
[22]: # MySQL connection
username = "root"
password = "12%40bittu"
host = "localhost"
port = "3306"
database = "customer_behavior"

engine = create_engine(f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}")


table_name = "mytable"
df.to_sql(table_name, engine, if_exists="replace", index=False)

# Read back sample
pd.read_sql("SELECT * FROM mytable LIMIT 5;", engine)
```

[22]:

	customer_id	age	gender	item_purchased	category	purchase_amount	location	size	color	season	review_rating	subscription_status	shipping_type	discount_applied	previous_purchases	pa
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	14	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	2	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	23	

4. Analysed the data using SQL

I performed structured analysis in MY SQL to answer key business questions:

1) Q1. What is the total revenue generated by male vs. female customers?

```
9  select gender, sum(purchase_amount)
10 from mytable
11 group by gender;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	age	gender	item_purchased	category	purchase_amount	location	size	color	season	review_rating	subscription_status	shipping_type	discount_applied	previous_purchases	payment_
1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	14	Venmo
2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	2	Cash
3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	23	Credit Car
4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	49	PayPal
5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	31	PayPal

2) Q2. Which customers used a discount but still spent more than the average purchase amount?

```
15 SELECT customer_id, purchase_amount
16 FROM mytable
17 WHERE discount_applied = 'Yes' AND purchase_amount >= ( SELECT AVG(purchase_amount) FROM mytable );
18
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	purchase_amount
2	64
3	73
4	90
7	85
9	97
12	68
13	72
16	81
20	90
22	62
24	88
29	94
32	79
33	67
35	91

mytable 2

3) Q3. Which are the top 5 products with the highest average review rating?

```
SELECT item_purchased, ROUND(AVG(review_rating), 2) AS "Average Product Rating"
FROM mytable
GROUP BY item_purchased
ORDER BY AVG(review_rating) DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	Average Product Rating
Gloves	3.86
Sandals	3.84
Boots	3.82
Hat	3.8
Skirt	3.78

4) **Q4. Compare the average Purchase Amounts between Standard and Express Shipping.**

```
SELECT shipping_type, ROUND(AVG(purchase_amount), 2) AS average_purchase_amount
FROM mytable
WHERE shipping_type IN ('Standard', 'Express')
GROUP BY shipping_type;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

shipping_type	average_purchase_amount
Express	60.48
Standard	58.46

5) **Q5. Do subscribed customers spend more? Compare average spend and total revenue between subscribers and non-subscribers.**

```
SELECT subscription_status, COUNT(customer_id) AS total_customers, ROUND(AVG(purchase_amount), 2) AS avg_spend, ROUND(SUM(purchase_amount), 2) AS total_revenue
FROM mytable
GROUP BY subscription_status
ORDER BY total_revenue DESC, avg_spend DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	total_customers	avg_spend	total_revenue
No	2847	59.87	170436
Yes	1053	59.49	62645

6) **Q6. Which 5 products have the highest percentage of purchases with discounts applied?**

```sql
SELECT item_purchased, ROUND(100 * SUM(CASE WHEN discount_applied = 'Yes' THEN 1 ELSE 0 END) / COUNT(*), 2) AS discount_rate
FROM mytable
GROUP BY item_purchased
ORDER BY discount_rate DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

item_purchased	discount_rate
Hat	50.00
Sneakers	49.66
Coat	49.07
Sweater	48.17
Pants	47.37

7) Segment customers into New, Returning, and Loyal based on their total number of previous purchases, and show the count of each segment.

```sql
WITH customer_type AS ( SELECT customer_id, previous_purchases, CASE WHEN previous_purchases = 1 THEN 'New'
WHEN previous_purchases BETWEEN 2 AND 10 THEN 'Returning' ELSE 'Loyal' END AS customer_segment
FROM mytable)

SELECT customer_segment, COUNT(*) AS "Number of Customers"
FROM customer_type
GROUP BY customer_segment;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_segment	Number of Customers
Loyal	3116
Returning	701
New	83

8) Q8. What are the top 3 most purchased products within each category?

```sql
WITH item_counts AS ( SELECT category, item_purchased, COUNT(customer_id) AS total_orders,
ROW_NUMBER() OVER ( PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
FROM mytable
GROUP BY category, item_purchased)

SELECT item_rank, category, item_purchased, total_orders
FROM item_counts
WHERE item_rank <= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

item_rank	category	item_purchased	total_orders
1	Accessories	Jewelry	171
2	Accessories	Sunglasses	161
3	Accessories	Belt	161
1	Clothing	Blouse	171
2	Clothing	Pants	171
3	Clothing	Shirt	169
1	Footwear	Sandals	160
2	Footwear	Shoes	150
3	Footwear	Sneakers	145
1	Outerwear	Jacket	163
2	Outerwear	Coat	161

Result 8

9) Q9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?

```sql
SELECT subscription_status, COUNT(customer_id) AS repeat_buyers
FROM mytable
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

subscription_status	repeat_buyers
Yes	958
No	2518

10) **Q10. What is the revenue contribution of each age group?**

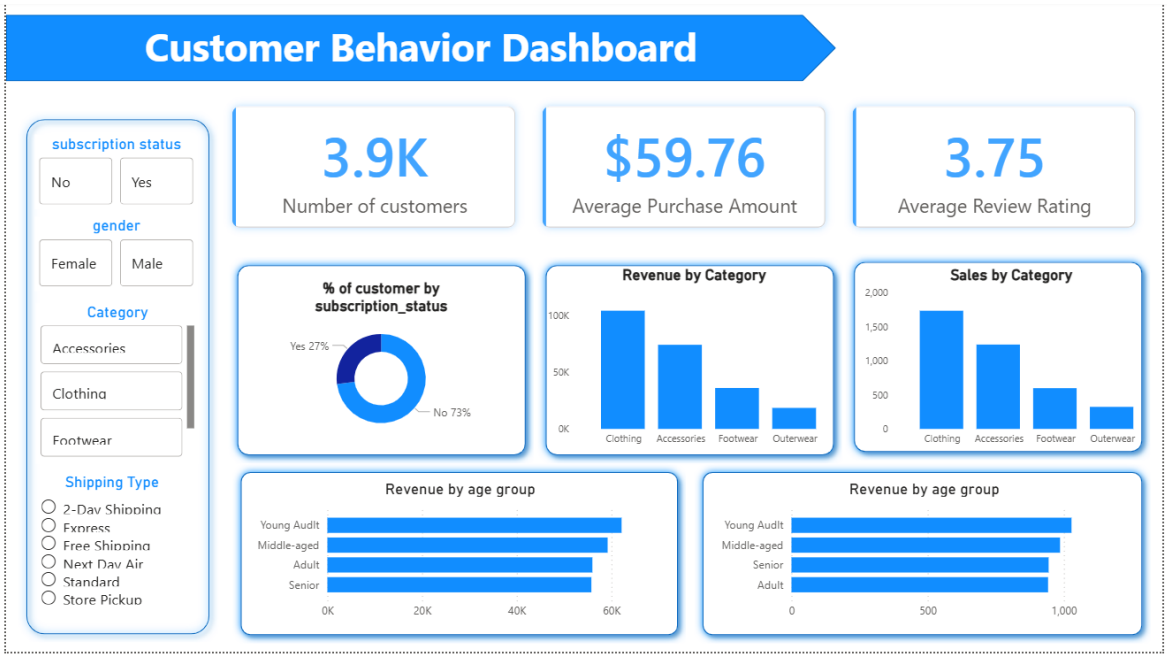
```sql
SELECT age_group, SUM(purchase_amount) AS total_revenue
FROM mytable
GROUP BY age_group
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

age_group	total_revenue
Young Audlt	62143
Middle-aged	59197
Adult	55978
Senior	55763

5. Dashboard in Power BI

Finally, I built an interactive dashboard in Power BI to present insights visually.



6. Business Recommendations

- Boost Subscriptions – Promote exclusive benefits for subscribers.
- Customer Loyalty Programs – Reward repeat buyers to move them into the “Loyal” segment.
- Review Discount Policy – Balance sales boosts with margin control.
- Product Positioning – Highlight top-rated and best-selling products in campaigns.
- Targeted Marketing – Focus efforts on high-revenue age groups and express-shipping users.